ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES · DUOC UC
ITY1102 — ARQUITECTURA DE SISTEMAS DE INTELIGENCIA ARTIFICIAL

Documentación de arquitectura (arc42)

MediFlow AI — Plataforma de agendamiento clínico asistida por IA

Evaluación Parcial 2 — 30%

Equipo: Zúñiga, Burgos, Cortez

Fecha: 26/10/2026

Tabla de contenidos

1. Introducción y objetivos 2
2. Restricciones 3
3. Alcance y contexto del sistema 3
4. Estrategia de solución 4
5. Vista de bloques de construcción 5
6. Vista de tiempo de ejecución 7
7. Vista de implementación 8
8. Conceptos transversales 10
9. Decisiones de arquitectura 12
10. Requisitos de calidad 12
11. Riesgos y deuda técnica 12
12. Glosario 12
Referencias 13
Declaración de uso de herramientas de IA generativa 13

Documento generado con la plantilla arc42 extendida. Los diagramas están elaborados en notación C4 con PlantUML, según la convención C1 del equipo.

1. Introducción y objetivos

1.1 Descripción general de los requisitos

MediFlow AI es una plataforma de agendamiento clínico que permite a un paciente reservar, reprogramar y cancelar horas médicas conversando en lenguaje natural, por el sitio web o por WhatsApp, sin formularios ni menús. El sistema interpreta la petición, consulta la disponibilidad real de los profesionales, registra la cita de forma transaccional e idempotente y notifica al paciente.

El propósito es reducir el tiempo de atención administrativa por reserva y liberar al personal de la clínica de la gestión repetitiva de agenda, manteniendo control humano sobre las excepciones. El uso es interno a la clínica y su personal administrativo, con acceso del paciente mediante un identificador de sesión de corta duración.

Características principales:

- Agente conversacional basado en un grafo de estados explícito (LangGraph) que decide qué herramienta invocar en cada turno.
- Consulta de especialidades, médicos y cupos libres contra la base de datos operacional, sin valores inventados por el modelo.
- Reserva, cancelación y reprogramación con control de concurrencia sobre el cupo y clave de idempotencia por solicitud.
- Desacoplamiento asíncrono entre la conversación y la ejecución, para absorber picos de demanda sin perder turnos.
- Observabilidad por turno: traza distribuida, tokens consumidos, herramientas ejecutadas y latencia por nodo del grafo.

Aspectos de capacidad considerados en el diseño: (i) el razonamiento del agente es el punto de mayor latencia y de mayor costo variable, por lo que se aísla en un servicio escalable de forma independiente; (ii) la escritura de citas debe ser exacta, no eventualmente consistente, y por eso queda en un único servicio transaccional; (iii) el sistema debe seguir respondiendo de forma degradada si el proveedor del modelo falla; (iv) el costo de infraestructura debe ser predecible frente a picos de hasta cinco veces la carga media.

1.2 Objetivos de calidad

Los atributos se expresan según ISO/IEC 25010.

Ítem	Atributo (ISO 25010)	Motivación
OC1	Eficiencia de desempeño — comportamiento temporal	El 95 % de los turnos debe responderse en menos de 8 s; un agendamiento lento se abandona.
OC2	Fiabilidad — tolerancia a fallos	Una caída del proveedor de LLM no debe impedir consultar disponibilidad ni derivar a humano.
OC3	Seguridad — confidencialidad e integridad	Datos de salud son sensibles: cifrado en tránsito y en reposo, y aislamiento por paciente.
OC4	Mantenibilidad — modificabilidad	Agregar una herramienta al agente no debe requerir desplegar la API ni el frontend.
OC5	Escalabilidad — capacidad de crecer	Sostener 5× la carga media sin rediseño, escalando solo el servicio que lo necesita.
OC6	Eficiencia de desempeño — utilización de recursos	Costo de infraestructura acotado y proporcional al uso, no al pico.

1.3 Interesados (Stakeholders)

Rol / Nombre	Objetivo / Alcance
Paciente	Agendar o reagendar en menos de dos minutos, sin llamar, y recibir confirmación verificable.
Personal administrativo	Mantener la agenda bajo control, resolver excepciones y ver el historial de cambios.
Dirección de la clínica	Reducir el costo por reserva y sostener la ocupación de horas sin sobrecargar al equipo.
Equipo de TI / Arquitecto	Operar, desplegar y monitorear la plataforma con trazabilidad y bajo riesgo de cambio.
Oficial de seguridad y privacidad	Cumplir la normativa de datos sensibles (Ley 19.628 y Ley 21.719) y demostrar minimización de datos.
Proveedor de LLM	Consumir la API dentro de cuotas y sin recibir datos personales identificables.

2. Restricciones

2.1 Restricciones técnicas

Restricciones de software y programación

Código	Restricción	Antecedente
RT1	Backend en Python 3.12 con FastAPI y LangGraph	Lenguaje y framework definidos en el EP1 y ya implementados en el repositorio del proyecto.
RT2	Base de datos relacional PostgreSQL 17	El modelo de agenda exige transacciones ACID y restricciones de exclusión temporal.
RT3	Frontend en React 18 sobre Vite	Base existente del proyecto; no se reescribe en esta etapa.

Restricciones del sistema operativo y plataforma

Código	Restricción	Antecedente
RT4	Contenedores Linux (imagen base slim) sobre Kubernetes	La pauta del proyecto exige orquestación con Kubernetes o ECS.
RT5	Región AWS sa-east-1 con dos zonas de disponibilidad	Latencia hacia Chile y requisito de alta disponibilidad multi-AZ.

Restricciones de hardware

Código	Restricción	Antecedente
RT6	Sin GPU en la ruta de inferencia: el modelo se consume como servicio externo	El costo de una GPU dedicada (USD 652–1.248/mes) no se justifica para el volumen proyectado; se reserva para fine-tuning fuera de línea.
RT7	Memoria por pod ≤ 1 GiB en el servicio de agente	Permite nodos de 16 GiB con densidad alta y reduce el costo de cómputo.

2.2 Restricciones organizacionales

Código	Restricción	Antecedente
RO1	Equipo de tres integrantes, 5 horas pedagógicas por evaluación	Define el alcance máximo de la entrega y el nivel de detalle documentable.
RO2	Entrega en PDF de 10 páginas con plantilla arc42 extendida	Formato institucional obligatorio de la evaluación parcial.

2.3 Convenciones

Código	Convención	Antecedente
C1	Diagramas de arquitectura en notación C4, generados con PlantUML	Versionables como texto y consistentes con la notación enseñada en la asignatura.
C2	Commits en formato Conventional Commits y ramas por funcionalidad	Trazabilidad entre requerimiento, cambio y despliegue.
C3	Documentación técnica y nombres de dominio en español; APIs y código en inglés	Coherencia con el dominio clínico y con las convenciones del ecosistema Python.

3. Alcance y contexto del sistema

Este capítulo describe el entorno del sistema: quién lo utiliza y de qué sistemas depende.

3.1 Contexto empresarial

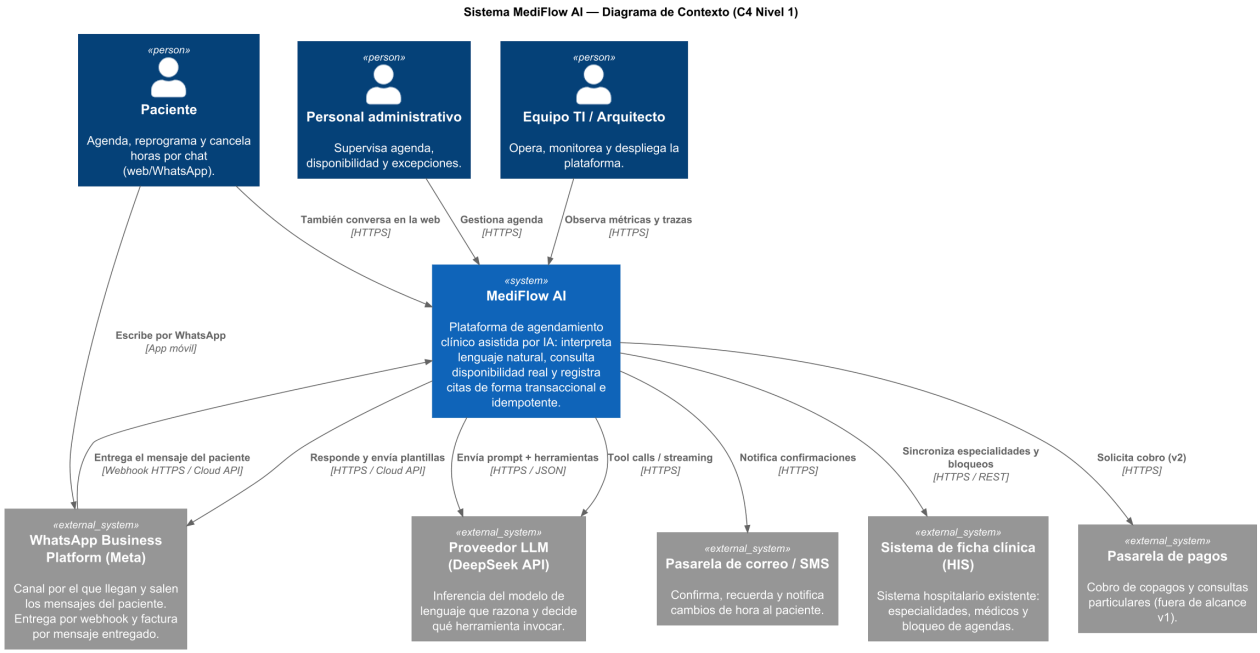


Figura — C1 Contexto

Vecino	Descripción
Paciente	Persona que reserva horas. Interactúa por web o WhatsApp; no accede a la operación interna.
Personal administrativo	Gestiona agenda, bloqueos y excepciones; es el humano en el circuito ante fallos del agente.
Proveedor de LLM (DeepSeek)	Entrega inferencia del modelo. Recibe el prompt con datos personales enmascarados.
Sistema de ficha clínica (HIS)	Fuente autoritativa de especialidades, médicos y bloqueos de agenda de la clínica.
Pasarela de correo / SMS	Entrega confirmaciones y recordatorios al paciente.
Pasarela de pagos	Cobro de copagos; fuera del alcance de esta versión.

3.2 Contexto técnico

El sistema se despliega en una VPC con subredes públicas y privadas en dos zonas de disponibilidad. El tráfico entra por CloudFront y WAF hacia un Application Load Balancer, que enruta a los servicios en Kubernetes. Dentro del clúster, el balanceo de carga lo hace un controlador de entrada NGINX (NGINX Ingress Controller): recibe del ALB el tráfico ya descifrado, aplica las reglas de enrutamiento por ruta, reparte las peticiones entre los pods con algoritmo de menos conexiones y reintentos, y mantiene las sesiones del canal. Las dependencias de datos viven en subredes privadas sin salida a internet; el único egreso permitido es hacia el proveedor de LLM, a través de NAT Gateway y del gateway de LLM. La topología completa de nodos y artefactos se detalla en la vista de implementación (capítulo 7).

Nodo / Artefacto	Descripción
CloudFront + WAF	Terminación TLS, reglas OWASP y limitación de tasa por IP.
Application Load Balancer	Distribución entre zonas; health check HTTP sobre /api/health cada 10 s.
NGINX Ingress Controller	Balanceo de carga L7 dentro del clúster: rutas por servicio, menos conexiones, reintentos y límites de tasa por ruta.
Cluster EKS	Orquestación de los servicios; tres node groups con perfiles de carga distintos.
RDS PostgreSQL Multi-AZ	Instancia primaria y standby con failover automático; réplica de lectura para consultas.
ElastiCache Redis Multi-AZ	Estado de sesión, checkpoints del grafo y bloqueos de idempotencia.
Secrets Manager + KMS	Credenciales y claves, inyectadas en runtime; nunca en la imagen ni en variables planas.

4. Estrategia de solución

La solución se apoya en cuatro decisiones estructurales que explican el resto del diseño.

- Separar el razonamiento del registro. El agente decide y conversa; un servicio distinto, con un único punto de escritura, registra la cita. Así la corrección transaccional no depende del comportamiento probabilístico del modelo.
- Asincronía entre canal y ejecución. La API responde 202 al recibir el mensaje y el turno se procesa desde una cola. El paciente puede escribir mientras el agente trabaja, y un pico de demanda se absorbe en cola en lugar de degradar la latencia.
- Estado explícito y durable. El grafo de LangGraph persiste un checkpoint por nodo en Redis, de modo que un pod que muere a mitad de una conversación puede reanudarla en el nodo siguiente sin repetir la reserva.

- Idempotencia de extremo a extremo. Cada solicitud de reserva lleva una clave y una huella del contenido; un reintento del cliente, del agente o de la red devuelve la misma cita en lugar de duplicarla.

Estas decisiones cubren los objetivos de calidad de mayor peso: OC1 y OC5 (aislar y escalar solo el componente que lo necesita), OC2 (degradación controlada si el LLM falla) y OC3 (el dato sensible no viaja al proveedor del modelo sin enmascarar).

4.1 Patrones de arquitectura aplicados

La solución es una arquitectura de microservicios con contenedores: cada capacidad (API de negocio, servicio de agente, agenda y reservas, gateway de LLM, adaptador de canal) se compila en su propia imagen, se despliega como Deployment independiente en EKS y escala por separado según su cuello de botella. La tabla resume los patrones que aparecen en el diseño, dónde se aplican y qué problema resuelven.

Patrón	Dónde se aplica	Qué resuelve
Microservicios con contenedores	api, agente, agenda-reservas, llm-gateway y adaptador de canal, cada uno en su imagen y su Deployment con HPA propio	Escalar solo el componente que lo necesita; aislar el fallo de un servicio del resto del sistema
API Gateway	Servicio api: autenticación, validación de esquema, limitación de tasa y enrutamiento a los servicios internos	Un único punto de entrada con políticas comunes, sin repetirlas en cada servicio
LLM Gateway (proxy con políticas)	Servicio llm-gateway entre el agente y el proveedor del modelo	Enmascarado de datos personales, timeout, reintentos con retroceso y circuit breaker por proveedor
Competing consumers	Consumidores de la cola SQS que alimenta al servicio de agente	Absorber picos en cola; varios pods compiten por los turnos y la latencia se mantiene acotada
Idempotencia y compensación (saga)	Cuerpo de la reserva y de la reagenda en agenda-reservas	Un reintento del modelo o de la red devuelve la misma cita en lugar de duplicarla
Cache-aside	Sesión, checkpoints del grafo y claves de idempotencia en vuelo en Redis	Evitar ir a la base en cada turno y permitir reanudar el grafo tras la caída de un pod
Circuit breaker	Cliente del proveedor de LLM y llamadas a servicios internos	Cortar la cascada de fallos y degradar a derivación humana en lugar de agotar los timeouts
Sidecar de telemetría	Colector de OpenTelemetry junto a cada servicio	Una traza por turno con spans por nodo del grafo y por herramienta, sin tocar el código de negocio

5. Vista de bloques de construcción

La descomposición se documenta en tres niveles C4: contenedores (nivel 1), componentes del servicio de agente (nivel 2) y organización del código (nivel 3).

5.1 Vista de bloques de construcción - Nivel 1

Diagrama de contenedores (caja blanca):

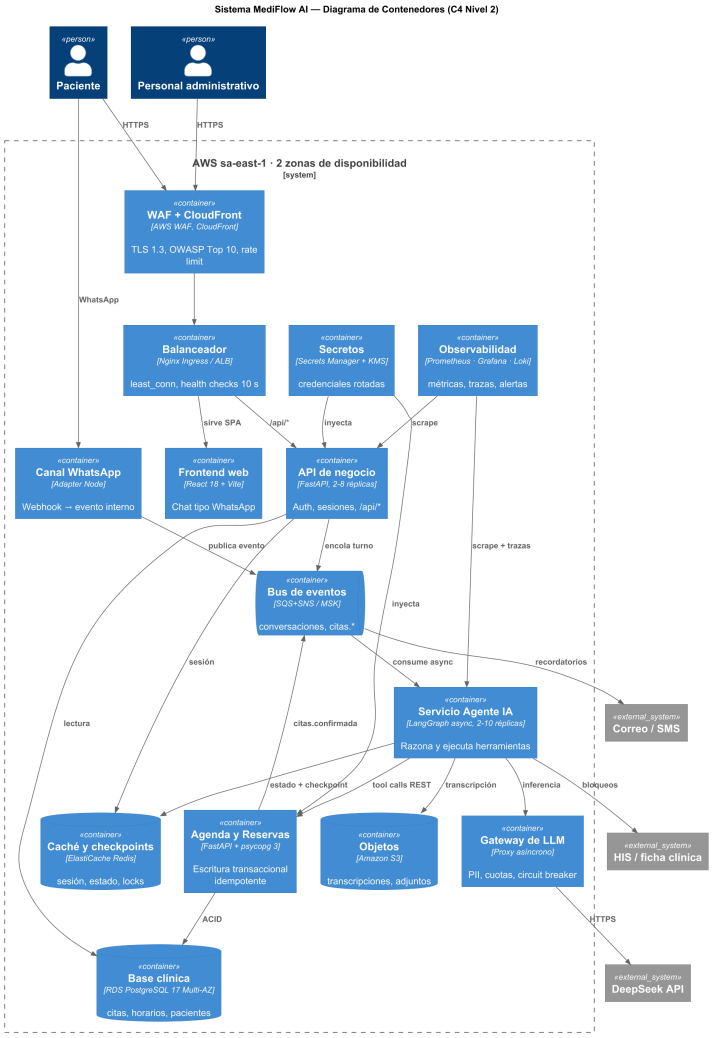


Figura — C2 Contenedores

Descripción de bloques (caja negra):

Bloque	Descripción
Frontend web	SPA React que simula la conversación de WhatsApp y permite confirmar la cita.
Canal WhatsApp	Adaptador que normaliza el webhook del proveedor a un evento interno y responde 200 en menos de 100 ms.
API de negocio	Autenticación, sesiones, validación de esquema y limitación de tasa; expone /api/chat, /api/citas y /api/horarios.
Servicio Agente IA	Grafo de estados que razona, elige herramientas y conserva el contexto de la conversación.
Agenda y Reservas	Única puerta de escritura de citas: reservar, cancelar y reagendar con bloqueo por cupo e idempotencia.
Gateway de LLM	Enmascara datos personales, aplica timeout, reintentos con retroceso y circuit breaker por proveedor.
Bus de eventos	Colas y temas que desacoplan conversación, registro y notificación.
Base clínica (RDS)	Esquema relacional de especialidades, médicos, horarios, pacientes y citas.
Caché y checkpoints (Redis)	Sesión, estado del grafo y claves de idempotencia en vuelo.
Observabilidad	Prometheus, Grafana y Loki con trazas OpenTelemetry.

5.2 Vista de bloques de construcción - Nivel 2

Diagrama de componentes del servicio de agente (caja blanca):

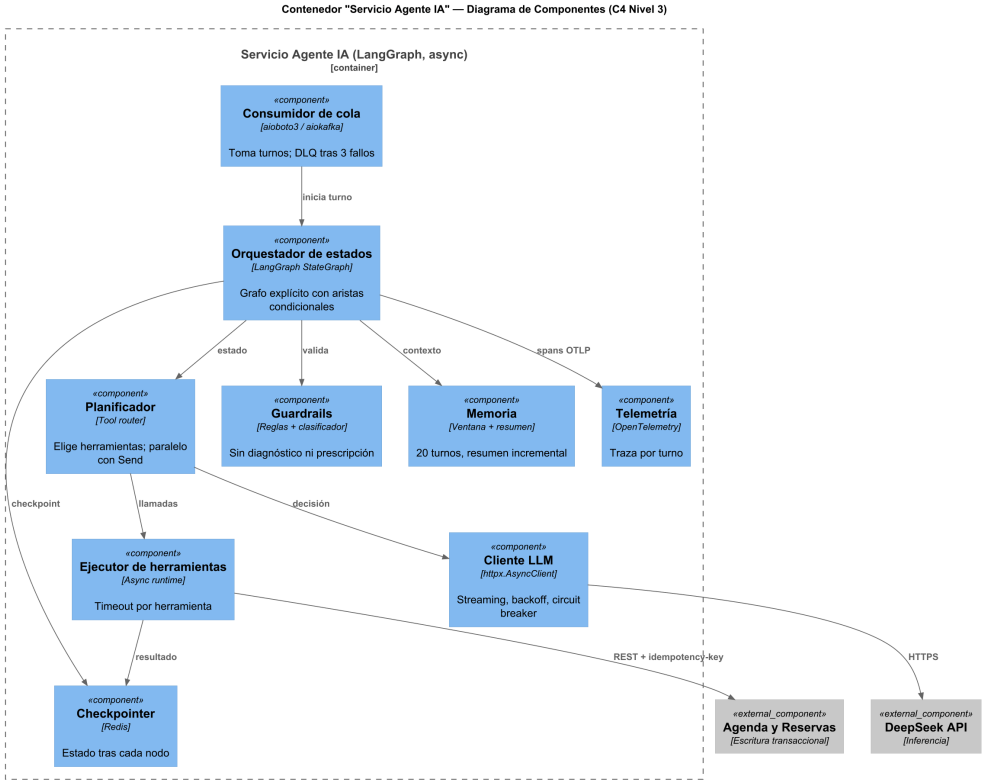


Figura — C3 Componentes

Descripción de componentes (caja negra):

Bloque	Descripción
Consumidor de cola	Toma turnos con visibilidad de 60 s; tras tres fallos deriva a cola de mensajes muertos.
Orquestador de estados	Grafo LangGraph: recibir → clasificar intención → planificar → ejecutar → validar → responder.
Checkpointer	Persiste el estado tras cada nodo para permitir reanudación ante caída del pod.
Planificador de herramientas	Selecciona la herramienta y admite llamadas paralelas cuando son independientes.
Ejecutor de herramientas	Invoca las herramientas con timeout individual y propaga la clave de idempotencia.
Guardrails de dominio	Bloquea diagnósticos y prescripciones y exige confirmación explícita antes de escribir.
Cliente LLM	Streaming, timeout de 20 s, retroceso exponencial y circuit breaker.
Telemetría	Una traza por turno con un span por nodo y por llamada a herramienta.

5.3 Vista de bloques de construcción - Nivel 3

El nivel de código se organiza por responsabilidad, no por tipo de archivo. La tabla resume los módulos del repositorio y su frontera.

Módulo	Responsabilidad
agente.py	Construcción del grafo, prompt de sistema y bucle de herramientas.
herramientas.py	Definición de las herramientas expuestas al modelo y su contrato de entrada.
backend/main.py	Superficie HTTP: chat, salud, especialidades, médicos y horarios.
backend/agenda.py	Consulta de cupos libres con zona horaria de Chile y exclusión de ocupados.
backend/reservas.py	Reserva, cancelación y reprogramación con idempotencia y bloqueo por cupo.
database/	Esquema inicial y migraciones versionadas.

6. Vista de tiempo de ejecución

Escenario principal: un paciente solicita una hora de dermatología y el agente la confirma. El diagrama muestra el camino completo, incluida la escritura transaccional y la notificación.

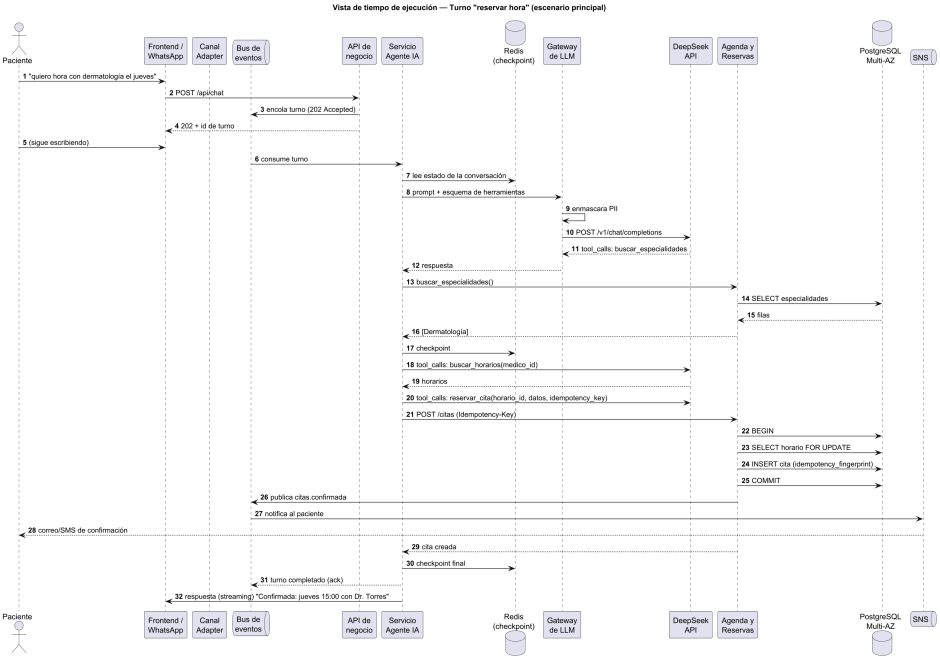


Figura — S1 secuencia reserva

Puntos relevantes del escenario: la API responde antes de que el turno se procese (202), el estado se persiste entre llamadas al modelo, y la reserva se ejecuta en una transacción con SELECT ... FOR UPDATE sobre el cupo. Si el proveedor del modelo falla, el consumidor reencola con retroceso y, agotados los reintentos, el turno va a la cola de mensajes muertos mientras el paciente recibe una derivación a atención humana.

7. Vista de implementación

Diagrama de despliegue completo (nodos, zonas y artefactos):

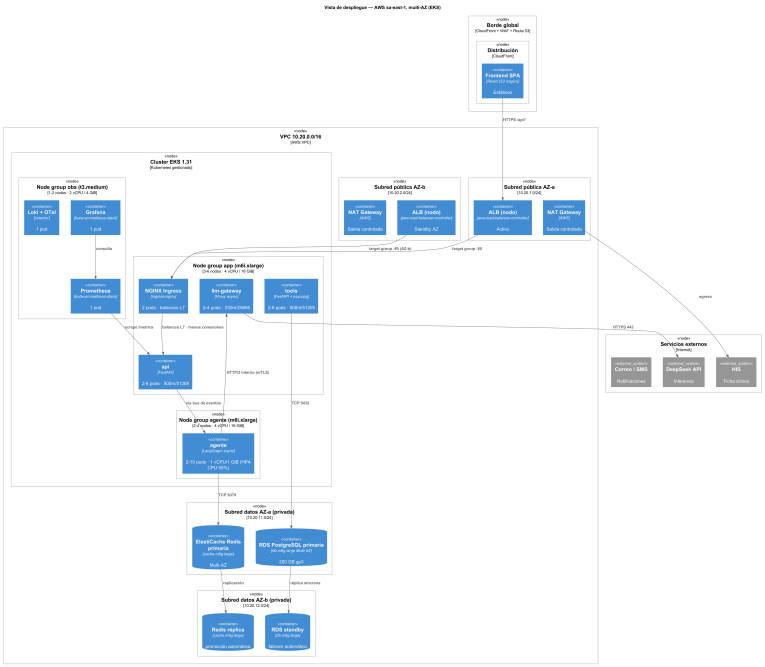


Figura — D1 despliegue

Nodo / Artefacto	Descripción
Node group app (m6i.xlarge)	nginx-ingress (2 pods), api (2-8 pods), tools (2-6 pods) y llm-gateway (2-4 pods).
Node group agente (m6i.xlarge)	agente (2-10 pods) con HPA por CPU al 65 % y por longitud de cola.
Node group obs (t3.medium)	Prometheus, Grafana, Loki y colector OpenTelemetry.
RDS PostgreSQL	db.m6g.large Multi-AZ, 200 GB gp3, respaldo con recuperación a un punto en el tiempo de 7 días.
ElastiCache Redis	cache.m6g.large Multi-AZ con réplica y promoción automática.

7.1 Dimensionamiento de recursos y costos

El dimensionamiento parte de una carga de diseño de 300 conversaciones diarias con 8 turnos promedio (72.000 turnos al mes) y un pico de cinco veces la media. Los precios son on-demand de la región sa-east-1, consultados en la lista de precios pública de AWS el 25/10/2026.

Recurso	Especificación	Cantidad	USD/mes
EKS	cluster gestionado	1 × 730 h	73,00
EC2 app	m6i.xlarge (4 vCPU / 16 GiB)	3 × 730 h	670,14
EC2 agente	m6i.xlarge	2 × 730 h	446,76
EC2 observabilidad	t3.medium (2 vCPU / 4 GiB)	1 × 730 h	49,06
RDS instancia	db.m6g.large Multi-AZ	730 h	303,68
RDS almacenamiento	gp3 Multi-AZ 200 GB	200 GB	87,60
ElastiCache	cache.m6g.large Multi-AZ	2 × 730 h	309,52
ALB	1 balanceador + 10 LCU-h	730 h	105,12
NAT Gateway	1 por zona de disponibilidad	2 × 730 h	135,78
S3 + CloudFront	500 GB almacenados, 300 GB egreso	—	53,25
Colas, secretos y discos	SQS, Secrets Manager, EBS gp3 400 GB	—	63,29
Total estimado			2.297,19

La estimación anterior corresponde al diseño objetivo con alta disponibilidad. Para la validación temprana se cotizó además una línea base de piloto en AWS Pricing Calculator (us-east-2): una tarea ECS Fargate de 0,25 vCPU y 1 GB, RDS db.t4g.micro Single-AZ con 20 GB, un ALB, un NAT Gateway, una IPv4 pública, Secrets Manager y 1 GB de logs, por USD 78,55 al mes. Esa cifra permite validar el flujo con tráfico real a bajo costo, pero no es comparable con el diseño objetivo: no contempla segunda zona de disponibilidad, ni el canal de WhatsApp, ni la carga de 72.000 turnos. La diferencia entre ambos escenarios (USD 78,55 frente a USD 2.297,19) mide exactamente lo que cuesta la alta disponibilidad.

Red y almacenamiento: 300 GB de egreso mensual por CloudFront con caché de estáticos; 500 GB en S3 con transición a Glacier a los 90 días; 200 GB de base de datos con crecimiento proyectado de 4 GB/mes; 400 GB de discos gp3 para los nodos. CPU/GPU: el razonamiento se ejecuta en el proveedor del modelo, por lo que no se aprovisiona GPU en la ruta en línea; una instancia g4dn.xlarge (USD 0,894/h) queda disponible solo para fine-tuning fuera de horario. RAM: 1 GiB por pod de agente, 512 MiB por pod de API, 16 GiB por nodo.

El costo de infraestructura equivale a USD 0,0319 por turno. La escalabilidad es horizontal y selectiva: duplicar la carga agrega nodos al grupo del agente y pods de API, pero no multiplica la base de datos ni el balanceador, que son el grueso del costo fijo. El controlador NGINX Ingress no suma cómputo: sus dos pods corren dentro del node group app ya dimensionado.

7.2 Costo del canal: WhatsApp Business Platform

La infraestructura no es el único costo variable del sistema: el canal por el que llegan las conversaciones también se cobra por mensaje. Desde el 1 de octubre de 2026 Meta factura los mensajes de servicio (las respuestas del bot dentro de la ventana de 24 horas) a la tarifa de utilidad del país, con 1.000 mensajes de servicio gratuitos por número y mes, sin arrastre. Para Chile la tarifa publicada es USD 0,0200 por mensaje entregado en las categorías de servicio, utilidad y autenticación, y USD 0,0889 en marketing.

Concepto	Volumen mensual	Tarifa	USD/mes
Mensajes de servicio (respuestas del bot)	72.000	0,0200 (1.000 gratis)	1.420,00
Recordatorios proactivos (plantilla utility)	2.250	0,0200	45,00
Mensajes entrantes del paciente	72.000	sin costo	0,00
Total del canal			1.465,00

El total del sistema queda en USD 3.762,19 al mes: USD 2.297,19 de infraestructura y USD 1.465,00 de mensajería. El canal representa el 39 % del costo, de modo que el costo real por turno sube de USD 0,0319 a USD 0,0523. Esta es la decisión de dimensionamiento más importante del capítulo: optimizar pods no mueve la aguja si el volumen de mensajes no se controla.

Palancas de reducción aplicadas al diseño: (1) agrupar la respuesta en un solo mensaje en lugar de varios, lo que reduce el conteo facturable sin empeorar la experiencia; (2) no usar plantillas de marketing para confirmaciones —la tarifa de marketing es 4,4 veces la de servicio—; (3) aprovechar la ventana de 72 horas sin costo cuando la conversación se inicia desde un anuncio con click-to-WhatsApp; y (4) derivar al CRM de recepción las conversaciones que superan cinco turnos, porque cada turno adicional se factura.

7.3 Contenedorización y Dockerfile de referencia

Cada servicio se empaqueta en su propia imagen y se publica en Amazon ECR. El repositorio incluye el Dockerfile del servicio de API y agente, construido en dos etapas: una etapa de dependencias que instala los paquetes en un prefijo aislado y una etapa

final que solo copia el resultado, de modo que la imagen entregada no lleva compiladores, caché de pip ni credenciales de construcción.

Decisión de la imagen	Valor aplicado	Motivo
Imagen base	python:3.12-slim	Misma versión que el entorno de desarrollo y pruebas; slim reduce superficie de ataque.
Etapas	deps y runtime	La imagen final no incluye pip ni caché; se reconstruye rápido en CI.
Usuario	appuser, uid 10001, sin shell	Un proceso comprometido no corre como root ni abre sesión interactiva.
Healthcheck	GET /api/health cada 15 s	Es el mismo endpoint que consulta el ALB: Kubernetes y el balanceador observan la misma señal.
Procesos	uvicorn con 2 workers; réplicas 2-8	El escalado se hace con pods (HPA), no con procesos dentro del contenedor.
Registro y versionado	ECR con etiqueta por commit y despliegue por digest	El tag mutable no garantiza qué se desplegó; el digest sí.
Escaneo	Trivy en CI, bloqueo si hay vulnerabilidades críticas	La imagen no se promueve a producción con un CVE crítico abierto.

La configuración no viaja dentro de la imagen: se inyecta por variables de entorno desde Secrets Manager y ConfigMaps. La misma imagen se usa en el entorno local, en el clúster de pruebas y en producción; lo único que cambia es la configuración, lo que evita el clásico "en mi máquina funciona". La imagen construida pesa 360 MB, arranca en 16 segundos con el healthcheck en estado sano, sirve el simulador en /app, el CRM en /crm y responde un turno completo con herramienta real en 3,2 segundos.

8. Conceptos transversales

8.1 Modelo de dominio

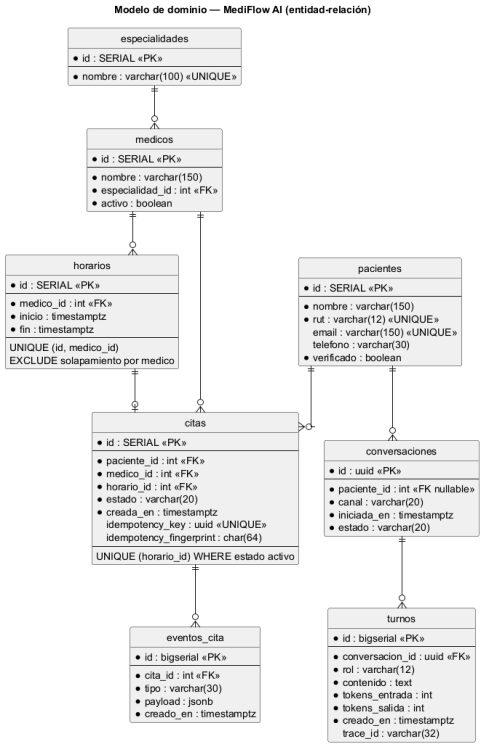


Figura — ER modelo dominio

Entidad / Tabla	Descripción
especialidades	Catálogo de especialidades médicas, único por nombre.
medicos	Profesional con especialidad y estado activo; los inactivos no se ofrecen.
horarios	Cupo con inicio y fin; una restricción de exclusión impide solapamientos por médico.
pacientes	Datos mínimos de contacto; el correo es único y el teléfono identifica al paciente sin correo.
citas	Reserva con estado, clave de idempotencia y huella de la solicitud.
conversaciones / turnos	Historial de la conversación y consumo de tokens por turno.
eventos_cita	Bitácora de cambios de estado publicados al bus de eventos.

8.2 Plan de seguridad

El plan sigue los diez puntos de la plantilla institucional de seguridad para sistemas de IA, con tres elementos aplicados al caso por punto.

Fase	Punto	Elementos aplicados e implementación
I. Gobierno	1. Políticas y clasificación de riesgos	Dato de salud como sensible (Ley 19.628 / 21.719); inventario de activos con dueño; matriz de riesgo revisada trimestralmente.
	2. Modelado de amenazas para IA	Inyección de prompt, abuso de la herramienta de reserva y fuga entre pacientes; STRIDE por contenedor con control y prueba por amenaza.
	3. Confianza cero	mTLS entre servicios; autorización por petición, no por red; roles IAM de mínimo privilegio.
	4. Cadena de suministro de datos	Procedencia del cupo desde el HIS con sello de tiempo; validación de esquema; migraciones versionadas; RUT validado con módulo 11 en la puerta de escritura, con clave única que impide dos fichas para la misma persona.
	5. Privacidad por diseño	Minimización (nombre, RUT y un contacto); el RUT se guarda normalizado y nunca se envía al proveedor del modelo; seudonimización en logs; transcripciones con retención de 90 días.
	6. Robustez y pruebas adversarias	60 casos adversarios en integración continua; criterio de fuga cero antes de desplegar.
III. Operación	7. Guardrails de entrada y salida	Filtro de intención fuera de dominio; prohibición de diagnóstico y prescripción; validación de salida contra esquema.
	8. Aislamiento de entornos	Namespaces y cuentas AWS separadas; contenedores sin privilegios y raíz de solo lectura; egreso solo al gateway de LLM.
	9. Monitoreo de deriva	Deriva de intención, de uso de herramientas y de latencia; tablero semanal en Grafana.
	10. Red teaming y respuesta	Ejercicio trimestral; runbook de incidentes; análisis posterior con acciones registradas.

Estado de los guardrails en el prototipo. Los puntos 6 y 7 no quedaron solo en el diseño: el servicio implementa ocho patrones de entrada (cambio de rol, pedido del prompt, formatos forzados, jailbreak) y cinco de salida (prompt, identificadores internos, claves, RUT y respuestas vacías). Un mensaje que activa un patrón de entrada se responde con una frase canónica sin llegar a gastar una llamada al modelo, y un contador visible en /api/guardrails registra cada bloqueo para el tablero. La batería de 20 ataques de tests/verificar_agente.py se ejecutó siete veces (140 turnos): cero fugas, cero respuestas vacías y cero errores HTTP, con los 18 intentos de manipulación detenidos en la capa determinista. El pedido de citas de otro número se rechaza porque la identidad queda fijada por el canal y no por el texto del mensaje.

La ficha del paciente se identifica por RUT: el agente lo pide al agendar y la herramienta lo normaliza y valida con el módulo 11 antes de escribir (acepta con puntos, sin puntos, con o sin guion y con K minúscula; rechaza el dígito verificador incorrecto sin tocar la base y vuelve a pedirlo). Un índice único impide dos fichas para la misma persona, y si el RUT aparece asociado a otro número la reserva se deriva a recepción en lugar de mezclar historiales. Diecisiete pruebas unitarias cubren los formatos y los rechazos.

Autenticación: OTP de un uso por correo o SMS y sesión con token firmado de 30 minutos; entre servicios, mTLS con certificados rotados por el mesh. Autorización: RBAC más control por recurso, de modo que una cita solo es visible para su paciente. Encriptación: TLS 1.3 en el borde y mTLS interno, KMS con clave propia para RDS, S3 y discos, y cifrado en reposo en Redis. Secretos: Secrets Manager con rotación a 30 días e inyección por CSI driver.

8.3 Observabilidad

Capa	Métrica	Umbral de alerta
Borde	p95 de latencia y tasa 5xx	p95 > 800 ms; 5xx > 1 %
API	Tasa de error y duración por endpoint	error > 2 % en 5 min
Agente	Latencia por nodo, herramientas por turno, tokens	turno p95 > 8 s; herramientas/turno > 4
LLM	Cuota, errores 429/5xx, latencia del proveedor	429 > 5 % en 10 min
Datos	Conexiones activas, lag de réplica	lag > 10 s
Negocio	Citas confirmadas por hora, conversaciones abandonadas	caída > 40 % vs. media móvil de 7 días

Cada turno genera una traza con un span por nodo del grafo y por herramienta ejecutada; los logs se emiten en JSON con el identificador de traza y las alertas se definen sobre presupuesto de error del objetivo de nivel de servicio, no sobre umbrales aislados.

8.4 Alta disponibilidad

El sistema opera en dos zonas de disponibilidad con RDS Multi-AZ (failover automático en menos de 60 s), Redis con réplica y promoción automática, y balanceador con health checks que retira nodos no sanos. Objetivos: RPO de 5 minutos (respaldos continuos y transacción síncrona a la standby) y RTO de 30 minutos (failover automático más despliegue desde las imágenes versionadas). Los respaldos se prueban mensualmente con una restauración real en un entorno aislado.

9. Decisiones de arquitectura

ID / Decisión	Problema y alternativas	Justificación
ADR-01 Ciclo del agente: grafo de estados versionado frente a cadena de llamadas al modelo	El ciclo razonar-actuar debe ser explícito, inspeccionable y reanudable tras la caída de un pod, no una sucesión de llamadas encadenadas en el código de la API.	Se implementa como grafo de estados (LangGraph) con nodos y aristas declarados en el repositorio, checkpoint por nodo en Redis y puntos de pausa para intervención humana. Una cadena de llamadas ad hoc no permite reanudar en el nodo siguiente ni auditar qué decidió el sistema en cada paso.
ADR-02 Orquestación: EKS frente a ECS	Se requiere escalado por servicio, políticas de red y observabilidad madura.	EKS aporta HPA por CPU y por métrica de cola, NetworkPolicy de egreso y despliegues progresivos; el plano de control (USD 73/mes) es marginal en el total.
ADR-03 PostgreSQL frente a NoSQL	La agenda exige excluir solapamientos y resolver reservas concurrentes del mismo cupo.	ACID, restricción de exclusión GiST, SELECT FOR UPDATE e índice único parcial garantizan una sola cita activa por cupo; en NoSQL esa garantía quedaría en la aplicación.
ADR-04 SQS/SNS frente a Kafka	72.000 turnos al mes con picos de 5×; no se requiere retención de flujo ni reprocesamiento.	SQS y SNS dan desacoplamiento, reintentos y cola de mensajes muertos sin operar un clúster; Kafka no se justifica con este volumen.
ADR-05 Idempotencia en la reserva	El agente puede reintentar y el paciente puede reenviar, duplicando citas.	Clave de idempotencia más huella SHA-256 del contenido, con índice único parcial: el reintento idéntico devuelve la misma cita y una clave reutilizada con otro contenido se rechaza.

10. Requisitos de calidad

Atributo	Escenario en el contexto del sistema
Desempeño	Con 5× la carga media (1.500 conversaciones/día), el turno p95 se mantiene bajo 8 s porque el servicio de agente escala de 2 a 10 pods y los turnos esperan en cola en lugar de agotar conexiones.
Disponibilidad	Si el proveedor de LLM deja de responder, la consulta de disponibilidad sigue operando y el paciente recibe derivación a atención humana; el sistema no pierde el turno.
Integridad	Si dos pacientes confirman el mismo cupo simultáneamente, solo una cita se registra y la segunda recibe el siguiente cupo disponible.
Seguridad	Ningún dato personal identificable del paciente llega al proveedor del modelo: el gateway enmascara nombre, correo, teléfono y RUT antes de enviar el prompt.
Mantenibilidad	Agregar una herramienta de negocio requiere un módulo nuevo y su registro en el grafo, sin desplegar API ni frontend.
Recuperación	Ante la caída de una zona de disponibilidad, el sistema se restablece en menos de 30 minutos con pérdida máxima de 5 minutos de datos.

11. Riesgos y deuda técnica

Riesgo	Tipo	Mitigación aplicada en el diseño
Latencia del proveedor de LLM	Técnico	Circuit breaker, caché de respuestas frecuentes y mensaje de espera activa: es el único componente fuera de control del equipo.
Costo variable por turno	Técnico	Ventana de 20 turnos con resumen incremental, límite de herramientas por turno y agrupación de la respuesta en un solo mensaje.
Razonamiento erróneo del agente	Técnico	El agente nunca escribe sin validar el identificador de cupo contra la base; la reserva se rechaza si el cupo no existe.
Cambio de versión del modelo	Técnico	Versión fijada en configuración y suite de regresión conversacional antes de actualizar.
Dependencia de un solo proveedor	Negocio	Gateway con adaptador intercambiable: el contrato se probó con dos proveedores.
Baja adopción del canal	Negocio	Convivencia con el canal telefónico y el CRM de recepción durante la marcha blanca.
Cumplimiento normativo	Negocio	Registro de consentimiento, base de licitud y bitácora de accesos a datos sensibles.

12. Glosario

Término	Descripción
C4	Modelo de diagramas de arquitectura en cuatro niveles: contexto, contenedores, componentes y código.
Idempotencia	Propiedad por la cual repetir una solicitud produce el mismo efecto que ejecutarla una vez.
Checkpoint	Estado persistido del grafo tras cada nodo, que permite reanudar una conversación.
HPA	Horizontal Pod Autoscaler: escalado automático de réplicas según métricas.
RPO / RTO	Pérdida máxima de datos aceptable y tiempo máximo de restablecimiento.
Multi-AZ	Despliegue en dos zonas de disponibilidad para tolerar la caída de una.
Guardrail	Control que restringe la entrada o la salida del modelo para evitar usos indebidos.

Referencias

Brown, S. (2018). The C4 model for visualising software architecture. Leanpub.

Starke, G., y Hruschka, P. (2022). arc42: Template for software architecture communication. https://arc42.org

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., y Wilkes, J. (2016). Borg, Omega, and Kubernetes. Communications of the ACM, 59(6), 50–57.

Kleppmann, M. (2017). Designing data-intensive applications. O'Reilly Media.

Organización Internacional de Normalización. (2023). ISO/IEC 25010: Sistemas y software: modelos de calidad del producto.

Amazon Web Services. (2026). Amazon EKS, Amazon RDS y precios on-demand de la región sa-east-1. https://aws.amazon.com/pricing/

Meta. (2026). Pricing on the WhatsApp Business Platform: tarifas por mensaje y niveles vigentes desde el 1 de octubre de 2026. Meta for Developers. https://developers.facebook.com/docs/whatsapp/pricing

Declaración de uso de herramientas de IA generativa

Se utilizaron herramientas de IA generativa como apoyo para la redacción y el ordenamiento de la documentación, la generación de los diagramas PlantUML a partir de las decisiones tomadas por el equipo y la verificación de precios de AWS. Las decisiones de arquitectura, el dimensionamiento, los patrones seleccionados y la validación de cada sección fueron revisados y contrastados por los integrantes del equipo, quienes asumen la responsabilidad del contenido entregado. Los precios citados provienen de la lista de precios pública de AWS consultada el 25/10/2026.